

CS 5326 · PA1: The Modern Transformer LM

Complete Study Guide and Quiz Preparation

This guide covers every part of the assignment, Sections 1 to 7 of the manual, in the order the manual uses. For each component it explains **what** it is, **why** it is used, **how** it is computed, what the implementation contract requires, and how **your** implementation and experiments handled it.

The last part is a question bank written the way the course staff might write a quiz: conceptual questions, calculations, code-reading questions and "why did you do X" questions about your own approach. Every question has a full answer in the answer key.

Part	Contents
A	Big picture: what you built, grading, the from-scratch rules, repository and tokenizer (Sections 1–2)
B	Architecture: tensor conventions, initialization, Linear, Embedding, RMSNorm, SwiGLU, RoPE, softmax, attention, GQA, block, full LM, parameter count (Section 3)
C	Loss and optimization: cross-entropy, perplexity, AdamW, LR schedule, gradient clipping (Section 4)
D	Training infrastructure: data loader, memmap, checkpointing, gradient accumulation, execution order (Section 5)
E	Text generation: decoding, temperature, top-p (Section 6)
F	Training on TinyStories: budget, tuning, final evaluation, artifact, Chinchilla (Section 7)
G	Your approach and results: every number you should know
H	Formula sheet and glossary of key terms
I	Quiz question bank
J	Answer key

How to use this guide

Read Parts B–F to understand, Part G to remember your own numbers, Part H the night before, then do Part I without looking at Part J.

Part A — The big picture (Sections 1-2)

A.1 What the assignment asks

You implement a **decoder-only Transformer language model** from scratch with low-level PyTorch tensor operations, train it on **TinyStories**, then evaluate its perplexity and study how decoding settings change its generated text.

You implement	You run	You submit
Transformer LM (Linear, Embedding, RMSNorm, SwiGLU, RoPE, softmax, attention, GQA, block, LM); cross-entropy; AdamW; LR schedule; gradient clipping; data loader; checkpointing; training loop; generation	Short tuning runs; one final 10,000-update training run; standardized validation (perplexity); decoding study (temperature, top-p)	src/, tests/adapters.py, REPORT.md, report_assets/, final_model.pt (FP16, CPU, model weights only)

A.2 How it is graded

- **70%**: programming implementation, graded by **unit tests**. These are the public tests plus **hidden tests** that check any behaviour described in the manual through the same adapter interface.
- **30%**: REPORT.md (training evidence, hyperparameter study, final evaluation, generated samples, analysis).

Common trap

Passing all public tests does not guarantee full marks. Hidden tests can check any rule written in the manual, such as returning the same dtype from RoPE or building no autograd graph inside AdamW.step().

A.3 The from-scratch rule

You may **not** use anything from torch.nn, torch.nn.functional or torch.optim except:

- torch.nn.Parameter
- container classes: nn.Module, nn.ModuleList, nn.Sequential ...
- torch.nn.init (e.g. trunc_normal_)
- the torch.optim.Optimizer base class

Top-level torch functions (torch.exp, torch.einsum, torch.sigmoid, torch.logsumexp ...) are allowed.

But **if a function is itself a deliverable, you may not call a built-in version of it**: no

nn.Linear, nn.Embedding, F.softmax, F.cross_entropy, torch.log_softmax, F.scaled_dot_product_attention, torch.optim.AdamW, clip_grad_norm_. tests/test_restrictions.py checks this.

A.4 Repository and adapters

- **src/**: your implementation (one file per component in your case: linear.py, embedding.py, rmsnorm.py, silu.py, swiglu.py, rope.py, softmax.py, attention.py, transformer_block.py, transformer_lm.py, loss.py, adamw.py, schedule.py, gradient_clipping.py, data.py, checkpoint.py, generation.py, train.py).
- **tests/adapters.py**: the only test file you edit. It is **glue code only**: import your class, construct it, copy the supplied weights in, call it, return the result. No maths is allowed in the adapter.
- **tests/test_*.py**: public tests. Do not edit them.
- The environment is managed by **uv**: `uv sync --frozen`, `uv run pytest`, `uv run python file.py`.

A.5 Dataset and tokenizer

- **TinyStories** (Eldan and Li, 2023): short, simple, synthetic children's stories. It is deliberately narrow so a small model can learn coherent English.
- The training stream has **466,876,982 tokens** from 2,119,719 stories. The validation stream has **4,692,376 tokens** from 21,990 stories.
- The tokenizer is a fixed **8,192-token byte-level BPE**. You do not train it. It is loaded only to encode prompts and decode generated IDs.
- **Byte-level BPE**: text is treated as UTF-8 bytes, so any string can be encoded (no unknown-token problem), and frequent byte sequences are merged into larger tokens to shorten sequences.
- The special token `<|endoftext|>` (**EOT**) separates stories in the stream and tells the generator when to stop.
- Stories are concatenated as `encode(story1), EOT, encode(story2), EOT, ...` with no BOS token, no padding and no pre-cut windows.

Key idea

The embedding table and LM head are **not pretrained**: they are ordinary parameters initialized randomly and learned during training.

Part B — Transformer architecture (Section 3)

B.1 What a language model computes

Input: token IDs of shape **(batch_size, sequence_length)**. Output: **logits** of shape **(batch_size, sequence_length, vocab_size)**. The logits at position i parameterize $p(x_{i+1} | x_{1:i})$, the distribution of the next token given all tokens so far. One forward pass produces all n next-token predictions in parallel, and the causal mask ensures position i only uses tokens $1 \dots i$.

Key idea

forward() returns **logits, not probabilities**. Softmax is applied only where needed: inside the loss (as log-sum-exp) or when sampling.

```
def forward(token_ids, token_positions=None):
    x = token_embedding(token_ids)           # (B, n) -> (B, n, d_model)
    for block in blocks:
        x = block(x, token_positions=token_positions)
    x = final_norm(x)                         # final RMSNorm
    return lm_head(x)                        # (B, n, vocab_size) logits
```

B.2 The fixed configuration

Symbol	Variable	Value	Meaning
$ V $	vocab_size	8192	vocabulary size
$n_{\max}$	context_length	256	maximum sequence length
d_{model}	d_model	512	residual-stream width
L	num_layers	4	number of Transformer blocks
h_q	n_q_heads	16	query heads
h_{kv}	n_kv_heads	4	key/value heads
g	h_q/h_{kv}	4	query heads per KV head (group size)
d_h	d_{model}/h_q	32	head dimension
d_{ff}	d_ff	1344	SwiGLU hidden width
Θ	rope_theta	10,000	RoPE frequency base
ϵ_{norm}	norm_eps	1e-5	RMSNorm stability constant

- No biases anywhere. No dropout anywhere. **Untied** embeddings: the input embedding and LM head are separate matrices. No absolute position embeddings: position information enters only through RoPE inside attention.
- The constructor must still take all of these as arguments, because the tests build smaller models.

B.3 Tensor conventions and $y = xW^T$

PyTorch stores a linear weight as $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$. With the feature dimension last, a row vector $x \in \mathbb{R}^{1 \times d_{\text{in}}}$ maps to $y = xW^T$. Maths in the manual writes the same thing as $y = Wx$ with column vectors. **The tests inject weights in the (out_features, in_features) orientation**, so storing W^T instead of W breaks every test.

Batch-like dimensions: batch, sequence position and attention head are all dimensions where the same operation repeats. Writing functions that accept arbitrary leading dimensions (...) lets one implementation handle all of them. **einsum / einops.rearrange** name the axes, which makes shape mistakes easy to catch. **jaxtyping** annotations such as `Float[Tensor, "batch seq d_model"]` document shapes.

B.4 Initialization

Parameter	Initialization	Why
Linear weight	$N(0, \sigma^2)$ with $\sigma^2 = 2/(d_{in} + d_{out})$, truncated at $\pm 3\sigma$	keeps activation variance roughly constant through layers (Xavier/Glorot-style)
Embedding	$N(0, 1)$ truncated at ± 3	unit-scale input vectors
RMSNorm gain γ	all ones	starts as a pure normalization

Use `torch.nn.init.trunc_normal_`. **Truncation** removes rare extreme values that could destabilize early training. Bad initialization can cause vanishing or exploding gradients. Pre-norm Transformers are fairly robust to it, but it still affects speed.

B.5 Linear (3 pts) and Embedding (3 pts)

- **Linear**: subclass `nn.Module` and store W as `nn.Parameter` of shape $(out_features, in_features)$. forward: $x @ W.T$ for x of shape $(..., in_features)$. No bias. `nn.Linear` and `F.linear` are forbidden.
- **Embedding**: an `nn.Parameter` table E of shape $(num_embeddings, embedding_dim)$. forward: $E[token_ids]$. Integer indexing maps IDs of any shape $(...)$ to $(..., d_model)$ and is **differentiable with respect to the selected rows**. `nn.Embedding` and `F.embedding` are forbidden.

B.6 RMSNorm (4 pts) and pre-norm

Post-norm (original Transformer): LayerNorm is applied after the residual add. **Pre-norm** (used here): normalize the *input* of each sub-layer and add the sub-layer output back to an **unnormalized residual stream**. The clean identity path from the embedding to the output improves gradient flow and training stability. A final RMSNorm is needed after the last block, because the residual stream itself is never normalized.

```
u = x + Attention(RMSNorm_attn(x))
y = u + SwiGLU(RMSNorm_ffn(u))
```

RMSNorm: $\text{RMSNorm}(x)_\ell = \gamma_\ell \cdot x_\ell / \text{RMS}(x)$, with $\text{RMS}(x) = \sqrt{((1/d) \sum x_c^2 + \epsilon)}$.

- **vs LayerNorm**: no mean subtraction (no centering) and no bias, only a learned gain γ . This is cheaper and works as well in practice (LLaMA uses it).
- Normalizes only over the **last** dimension.
- **Precision rule**: if the input is float16/bfloat16, upcast to float32 for the computation (squaring small numbers can underflow in low precision), then **cast the result back to the input dtype**. float32/float64 inputs are kept as they are.
- ϵ prevents division by zero and stabilizes near-zero vectors.

B.7 SiLU and SwiGLU (4 pts)

SiLU / Swish: $\text{SiLU}(x) = x \cdot \sigma(x) = x / (1 + e^{-x})$. It is smooth everywhere (unlike ReLU's kink at 0) and allows small negative outputs. `torch.sigmoid` is allowed.

GLU (Dauphin et al.): $\sigma(W_{\text{gate}} x) \odot W_{\text{up}} x$, an element-wise **gate** that decides how much of each hidden feature passes through. **SwiGLU** (Shazeer 2020) uses SiLU as the gate:

```
SwiGLU(x) = W_down [ SiLU(W_gate x) \odot (W_up x) ]
W_gate, W_up: (d_ff, d_model)      W_down: (d_model, d_ff)      three bias-free matrices
```

- **Why $d_{\text{ff}} \approx (8/3) \cdot d_{\text{model}}$** : a classic 2-matrix FFN with width $4d$ has $2 \cdot d \cdot 4d = 8d^2$ parameters. SwiGLU has 3 matrices, so $3 \cdot d \cdot d_{\text{ff}} = 8d^2$ gives $d_{\text{ff}} = 8d/3 \approx 1365$ for $d = 512$. It is rounded to a **multiple of 64** for hardware efficiency: **1344 = 21 × 64**.
- Used in LLaMA 3, Qwen 2.5 and PaLM. Shazeer's paper says it works but offers no explanation ("divine benevolence").

B.8 RoPE — Rotary Positional Embeddings (6 pts)

RoPE injects position by **rotating** each query and key head vector by a position-dependent angle. **Values are never rotated.** RoPE has **no learnable parameters**.

- **Adjacent-pair convention** (used here): coordinate pairs $(x_1, x_2), (x_3, x_4), \dots$ are each rotated as a 2-D vector.
- Frequency of pair k : $\omega_k = \Theta^{-(2k-2)/d_h}$, $k = 1 \dots d_h/2$. Angle at position i : $\phi_{i,k} = i \cdot \omega_k$. Early pairs rotate fast (high frequency), later pairs slowly.
- Rotation of a pair: $x'_{2k-1} = x_{2k-1} \cos \phi - x_{2k} \sin \phi$; $x'_{2k} = x_{2k-1} \sin \phi + x_{2k} \cos \phi$.
- **Efficient form**: $R_i x = x \odot c_i + \text{rotate_pair}(x) \odot s_i$, with $\text{rotate_pair}(x) = (-x_2, x_1, -x_4, x_3, \dots)$. Never build the dense $d_h \times d_h$ block-diagonal matrix; it is mostly zeros.

Key idea

Why RoPE works: relative position. $R(\phi_i)^T R(\phi_j) = R(\phi_j - \phi_i)$, so $(R_i q)^T (R_j k) = q^T R_{j-i} k$. The query-key score depends only on the **offset $j - i$** , not on absolute positions.

- **Implementation contract**: precompute cos and sin tables of shape **(context_length, $d_h/2$)**, the compact form. Store them with `register_buffer(..., persistent=False)`: they move with the module across devices but are not trainable and are not saved in the state_dict.
- Index the tables with `token_positions`; leading dimensions broadcast over batch and heads.
- Reject: odd head_dim, a last dimension that does not match head_dim, non-integer positions, positions outside $[0, \text{context_length})$. Return the same shape **and dtype** as the input.
- **Half-split convention** (GPT-NeoX) pairs coordinate i with $i + d/2$ instead. It is the same family of rotations but a different layout. **Never mix the two conventions**; the tests use adjacent pairs.

Your bug fix

In your code, cos/sin are float32 buffers, so a float16 or bfloat16 input came back as float32. The fix: `return result.reshape_as(x).to(x.dtype)`.

B.9 Softmax (2 pts)

$\text{softmax}(z)_\ell = \exp(z_\ell) / \sum \exp(z_c)$. **Numerical stability**: $\exp$ of a large number overflows to `inf`, and `inf/inf = NaN`. Softmax is invariant to adding a constant, so subtract the maximum first: $\exp(z_\ell - z_{\max})$. The largest exponent becomes $\exp(0) = 1$, and nothing overflows.

B.10 Scaled dot-product attention (5 pts)

$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$
 $Q: (\dots, n_q, d_k) \quad K: (\dots, n_{kv}, d_k) \quad V: (\dots, n_{kv}, d_v) \rightarrow (\dots, n_q, d_v)$

- **Why divide by $\sqrt{d_k}$** : if q and k have independent unit-variance entries, $q^T k$ has variance d_k . Large scores push softmax into a nearly one-hot, saturated region with tiny gradients. Scaling keeps the score variance around 1.
- **Mask convention: True = allowed**, False = blocked. Blocked scores are set to $-\infty$ **before** softmax, so their probability is exactly 0.
- Validate that every query row has at least one allowed key; otherwise softmax is $0/0 = \text{NaN}$. Fail clearly instead of returning non-finite values.
- Leading batch dimensions and the mask must broadcast. $n_q \neq n_{kv}$ and $d_v \neq d_k$ must both work.
- Masking is much more efficient than computing attention on sub-sequences one at a time.

B.11 Causal grouped-query self-attention (10 pts — the largest item)

Causal masking: query i may attend to key j only if $j \leq i$; future tokens are blocked. Build it with `torch.tril` or by comparing positions. Without it, the model would see the answer during training and learn nothing useful for generation.

MHA vs GQA vs MQA:

Variant	KV heads	Group size g	Notes
Multi-head (MHA)	$h_{kv} = h_q$	1	each query head has its own K, V
Grouped-query (GQA)	$1 < h_{kv} < h_q$	h_q/h_{kv}	each K/V head is shared by g query heads (this model: 16 Q, 4 KV, $g = 4$)
Multi-query (MQA)	$h_{kv} = 1$	h_q	one K/V head shared by all query heads

- **Why GQA:** it keeps most of MHA's quality with fewer K/V parameters and a much smaller **KV cache** at inference time (4× smaller here). It interpolates between MHA and MQA (Ainslie et al., 2023).
- **Shapes:** W_Q is $(h_q d_h, d_{\text{model}}) = (512, 512)$; W_K, W_V are $(h_{kv} d_h, d_{\text{model}}) = (128, 512)$; W_O is $(d_{\text{model}}, h_q d_h) = (512, 512)$.
- **One matrix multiplication** each for Q, K and V. Reshape into heads, apply RoPE to Q and K (never V), then group Q as $[B, h_{kv}, g, n, d_h]$. Query head $a = (h-1)g + r$ belongs to KV head h .
- Scores: $S_{b,h,r,i,j} = (1/\sqrt{d_h}) \sum Q \cdot K$ via einsum `'bhrid,bhjd->bhrij'`. The einsum **broadcasts** each K/V head across its g query heads. **Do not materialize repeated K/V** (no `repeat_interleave`).
- After $A \cdot V$, merge (h, r) back into the original query-head order, reshape to (B, n, d_{model}) and apply W_O . Getting the (h, r) order wrong means W_O receives the heads in a different order and the test fails.

B.12 Transformer block (5 pts) and full LM (6 pts)

- **Block:** two independent RMSNorms, causal RoPE-GQA, SwiGLU and two residual additions. `token_positions` are passed unchanged to attention; SwiGLU is position-wise and does not use them.
- **LM:** token embedding $\rightarrow$ L blocks $\rightarrow$ final RMSNorm $\rightarrow$ untied, bias-free LM head. Store `self.context_length` so `generate()` can check it. Reject empty sequences and sequences longer than `context_length`. `token_positions=None` means positions 0 ... $n-1$.

B.13 Parameter count = 19,272,192 (know this derivation)

Total = $2 V d + L[2d^2 + 2 \cdot d \cdot h_{kv} \cdot d_h + 3 \cdot d \cdot d_{ff} + 2d] + d$			
Embedding + LM head	: $2 \times 8192 \times 512$	=	8,388,608
Per block:			
$W_Q + W_O$	: 2×512^2	=	524,288
$W_K + W_V$	: $2 \times 512 \times (4 \times 32)$	=	131,072
SwiGLU	: $3 \times 512 \times 1344$	=	2,064,384
two RMSNorm gains	: 2×512	=	1,024
block total		=	2,720,768
4 blocks	: $4 \times 2,720,768$	=	10,883,072
Final RMSNorm	: 512	=	512
TOTAL		=	19,272,192

Observations: the embedding and LM head together are **43.5%** of all parameters (a small model with a relatively large vocabulary). GQA saves $2 \times 512 \times 384 = \mathbf{393,216}$ parameters per block compared with MHA (K and V are 128 wide instead of 512).

Part C — Loss and optimization (Section 4)

C.1 Cross-entropy (3 pts)

Training objective: the average negative log-likelihood of each next token: $L = (1/|D|n) \sum -\log p_{\theta}(x_{i+1} | x_{1:i})$. For one position with logits z and target y :

```
loss(z, y) = -z_y + log Σ_c exp(z_c)          (the second term is log-sum-exp)
```

- Compute log-sum-exp with a **max shift**: $\log \sum \exp(z_c) = m + \log \sum \exp(z_c - m)$, with $m = \max z$. `torch.logsumexp` is allowed.
- **Pick the target logit directly** with `gather`. Do not build a full `log(softmax(logits))` tensor: it wastes memory ($B \times n \times 8192$ floats) and `log(softmax)` can give $\log(0) = -\text{inf}$.
- Average over **all** leading dimensions (batch and sequence).
- Forbidden: `torch.softmax`, `torch.log_softmax`, `F.cross_entropy`.
- **Units**: natural log, so the loss is in **nats per token**. A uniform guess over 8,192 tokens gives $\ln 8192 \approx 9.01$, the expected loss at initialization.

C.2 Perplexity

```
PPL = exp( (1/n) Σ loss_i ) = exp(mean cross-entropy)
```

- Interpretation: the **effective number of equally likely choices**. $\text{PPL} \approx 5$ means the model's average uncertainty equals choosing among about 5 equally likely tokens. It does **not** mean the model literally puts equal weight on 5 tokens.
- Lower is better. A random model has $\text{PPL} = 8,192$; a perfect model has $\text{PPL} = 1$.
- **Always average the cross-entropy first, then exponentiate once**. Averaging per-batch perplexities is wrong: $\exp$ is convex, so by Jensen's inequality $\text{mean}(\exp(\ell_b)) \geq \exp(\text{mean } \ell_b)$. It overestimates PPL and gives extra weight to hard batches.

C.3 AdamW (5 pts)

Adam keeps, for every parameter, a running mean of gradients (**first moment m** , like momentum) and of squared gradients (**second moment v**). Each coordinate takes a step of about $\alpha \cdot m / \sqrt{v}$, so it adapts its own effective learning rate. **AdamW** adds **decoupled weight decay**.

```
for each parameter p with a gradient g:
    t = t + 1                                # per-parameter step count
    m = β1·m + (1-β1)·g                      # first moment
    v = β2·v + (1-β2)·g²                     # second moment
    m_hat = m / (1 - β1^t) ; v_hat = v / (1 - β2^t) # bias correction
    p = (1 - α·λ)·p                          # decoupled weight decay
    p = p - α · m_hat / (√v_hat + ε)         # adaptive update
```

- **Bias correction**: m and v start at 0, so early estimates are biased toward 0. Dividing by $(1 - \beta^t)$ removes this bias. Without it, the first steps would be far too small (for m) or too large (for v , via the $\sqrt{v}$ denominator).
- **Decoupled vs L2**: adding $\lambda\theta$ to the gradient (L2 regularization) sends the decay through Adam's adaptive scaling, so parameters with large gradient history get less decay. AdamW applies $(1 - \alpha\lambda)\theta$ **directly** to the weights, so every weight decays at the same rate (Loshchilov and Hutter, 2019).
- **β values**: (0.9, 0.999) is the classic default; (0.9, 0.95) is common for large LMs (GPT-3, LLaMA). A smaller β_2 makes v react faster to changes in gradient scale. $\epsilon \approx 1e-8$ prevents division by zero.
- **Contract**: validate $\text{lr} \geq 0$, $\text{eps} \geq 0$, $\text{weight_decay} \geq 0$, $0 \leq \beta_1, \beta_2 < 1$ in the constructor **and** in every parameter group, both at construction and again inside `step()` (the lr may be changed later by the scheduler). Initialize state lazily: `step = 0`, `exp_avg = zeros_like(p)`, `exp_avg_sq =`

zeros_like(p). **Skip** parameters with grad None completely (no decay, no step increment). Reject sparse gradients. Use the per-parameter count t, not the global step. If a closure is given, evaluate it once with gradients enabled and return its loss. The state must survive state_dict()/load_state_dict() for exact resumption.

- **No autograd graph** during updates: decorate step() with @torch.no_grad().

Your bug fix

Your original step() did the moment updates outside no_grad, so a gradient that itself required grad left exp_avg with a grad_fn. The fix is @torch.no_grad() on step(); the closure still runs under torch.enable_grad().

C.4 Learning-rate schedule: linear warmup + cosine decay (2 pts)

$$\begin{aligned} \alpha_s &= (s / s_w) \cdot \alpha_{\max} & \text{if } s < s_w \\ \alpha_s &= \alpha_{\min} + \frac{1}{2} \cdot (1 + \cos(\pi \cdot (s - s_w) / (s_c - s_w))) \cdot (\alpha_{\max} - \alpha_{\min}) & \text{if } s_w \leq s \leq s_c \\ \alpha_s &= \alpha_{\min} & \text{if } s > s_c \end{aligned}$$

- **Warmup:** Adam's second-moment estimates are unreliable in the first steps, and a large LR on a random model can diverge. Warmup ramps the LR up gradually. Note that $\alpha_0 = 0$, so the very first update changes nothing.
- **Cosine decay:** large steps early for fast progress, small steps late to settle into a good minimum and reduce noise (LLaMA schedule).
- At the midpoint of the cosine phase the LR is exactly $(\alpha_{\max} + \alpha_{\min})/2$.
- **Contract:** support zero warmup; hold $\alpha_{\min}$ after s_c ; require $0 \leq \alpha_{\min} \leq \alpha_{\max}$; reject non-integer or negative steps, $s_w < 0$ and $s_w \geq s_c$.
- **Final run:** $N = 10,000$ updates, $s = 0 \dots 9999$, $s_c = 9999$, so the last update uses exactly $\alpha_{\min}$.

C.5 Global gradient clipping (2 pts)

$$\begin{aligned} \|g\| &= \sqrt{(\sum_p \sum_i g_i^2(p))} & \text{one norm over ALL parameters} \\ \text{if } \|g\| > M: & \text{ every } g \leftarrow g \cdot M / (\|g\| + \epsilon_{\text{clip}}) & (\epsilon_{\text{clip}} = 1e-6) \end{aligned}$$

- **Why:** an unusually large gradient (for example from a rare batch) can destabilize a whole update. Clipping caps the update size.
- **Global** (one norm for all parameters) keeps the **direction** of the full gradient; only its length changes. Per-parameter clipping would change the direction.
- Ignore parameters with grad None; return the norm **before** clipping as a Python float (0.0 if no gradients); reject $M \leq 0$. Clip after backward and before optimizer.step().

Part D — Training infrastructure (Section 5)

D.1 Opening the token stream: np.memmap (2 pts)

- The files are raw **little-endian uint16** IDs (dtype '`<u2`'), 2 bytes per token, with no header. uint16 is enough because $8,192 < 2^{16} = 65,536$.
- **np.memmap** maps the file into virtual memory; the OS loads only the pages you actually read. The 467M-token training file is about 934 MB on disk. Converting it to int64 would need about **3.7 GB** of RAM, which is why the manual forbids `np.asarray(train, dtype=np.int64)`.
- Open with **mode='r'** (read-only, so the data cannot be modified by accident). Reject missing files and files with an odd byte length (a whole number of 2-byte tokens is required).

D.2 Next-token batch sampling (4 pts)

```
sample start o uniformly from [0, N - n)      # torch.randint(0, N - n, ..., generator=g)
window = u[o : o + n + 1]                    # n + 1 tokens
x = window[:-1] ; y = window[1:]              # targets are inputs shifted by one
```

- **Why $N - n$ as the exclusive upper bound:** the largest start is $N - n - 1$, and its window ends at index $N - 1$, the last token. Sampling up to $N - n$ inclusive would read past the end. Reading only n tokens would leave no target for the last position.
- Read only the sampled slices, stack them and convert only those to `torch.long` on the requested device.
- Windows may cross story boundaries; the EOT token marks the transition, so the model learns to start a new story after EOT.
- Reproducibility comes from passing an explicit **torch.Generator** to `torch.randint`, not from resetting the global seed. Same seed $\Rightarrow$ same sequence of batches; successive calls advance to new batches.
- Require positive batch and sequence lengths and at least $n + 1$ tokens.

D.3 Checkpointing (4 pts)

- A checkpoint must contain everything needed to continue the **same trajectory**: model state_dict, optimizer state_dict (the moments and step counts), **next_step**, and the states of **both** generators (get_state / set_state).
- **next_step** = the next global step to execute. After completing step s , save $\text{next_step} = s + 1$. Before any update, $\text{next_step} = 0$.
- `load_checkpoint` restores everything and returns next_step , and the loop resumes from exactly that step.
- Save only at complete global-step boundaries (after all microbatches and the optimizer update).
- Your file sizes show what is inside: `final_checkpoint.pt` ≈ 231 MB = FP32 weights (77 MB) + `exp_avg` (77 MB) + `exp_avg_sq` (77 MB). `final_model.pt` ≈ 38.5 MB = $19.27\text{M} \times 2$ bytes (FP16).

D.4 Two independent generators

The training script creates and seeds **one generator for training batches and one for validation batches** (yours: seed 42 and seed 43). Keeping them separate means that changing how often you validate does **not** change the sequence of training batches. Runs with different `eval_interval` therefore still see identical training data.

D.5 Gradient accumulation

- A **global step** = one **optimizer update**, not one forward pass.
- Microbatch B sequences of length n ; accumulate over n_{acc} microbatches. Tokens per update = **B**
 - $n \cdot n_{\text{acc}} = 16 \times 256 \times 16 = \mathbf{65,536}$. Effective batch size = $B \cdot n_{\text{acc}} = \mathbf{256}$ sequences.
- Why: a larger effective batch without storing all its activations at once (memory).

- Divide each microbatch loss by n_{acc} before `backward()`. The summed gradients then equal the gradient of the **mean** loss over the full effective batch.
- `zero_grad` once at the start, `clip` once, `optimizer.step()` once. The LR schedule, `next_step` and the 10,000-update budget all count optimizer updates.
- The logged `train_loss` is the mean of the **unscaled** microbatch losses.

D.6 Canonical execution order

```
train mode → α_s → zero grads → n_acc microbatches (backward) → clip → optimizer.step
           → completed = step + 1 → VALIDATE → LOG → CHECKPOINT
```

- When validation and checkpointing fall on the same step: validation first, then logging, then checkpoint. The final update forces all three.
- Because the checkpoint is saved after validation, the saved val-generator state already includes that validation's batches. On resume, both generators continue exactly where they would have been.
- **evaluate_validation**: `save was_training` → `model.eval()` → `torch.inference_mode()` → average several independent validation batches → `model.train(was_training)`.
- **eval() vs inference_mode()**: `eval()` switches modules to evaluation behaviour (relevant for dropout or batch-norm; this model has neither); it does **not** disable autograd. `inference_mode()` disables gradient tracking, so no graph is built. This saves memory and time.
- Log at least: completed step, mean unscaled train loss, val loss (when computed), LR and **pre-clipping** gradient norm.
- **Sanity check before a long run**: repeatedly optimize one fixed minibatch with a small model. A correct implementation drives that loss down sharply (overfits), which proves the model, loss, backward and optimizer are connected correctly.

Part E — Generating text (Section 6)

E.1 Autoregressive decoding

Start from a non-empty **prompt**. The prompt must be non-empty because the model needs a final position whose logits give the first next-token distribution. Repeat: run the model, take the logits at the **last** position, turn them into a distribution, sample a token, append it. Stop at EOT or after `max_new_tokens`. Return prompt + completion.

```
for each new token:
  1. crop the sequence to its last context_length tokens      (sliding window)
  2. add batch dim, run model, take logits[0, -1]
  3. divide logits by temperature, stable softmax
  4. sort probs descending, keep the shortest prefix with cumulative ≥ top_p, zero the rest, renormalize
  5. torch.multinomial on the sorted distribution (with generator) → rank → sorted_indices[rank] = token ID
  6. append; stop immediately AFTER generating EOT (EOT is kept in the output)
```

- **Contract:** prompt is a non-empty 1-D torch.long tensor on the model's device; `max_new_tokens` ≥ 0 (0 returns the prompt unchanged); temperature > 0 ; $0 < \text{top_p} \leq 1$; the model must expose `context_length` and the argument must equal it (no silent cropping to a different window).
- Save the previous mode, call `model.eval()`, run the loop under `torch.inference_mode()`, and restore the mode before returning.
- There is **no KV cache** in this assignment, so each step recomputes the whole cropped context. This is simple but slow: $O(n)$ forward cost per new token.
- Generation is not unit-tested; it is graded through `REPORT.md`.

E.2 Temperature

$$\text{softmax}(z/\tau)_c = \exp(z_c/\tau) / \sum \exp(z_{c'}/\tau) \quad \tau > 0$$

- $\tau < 1$ **sharpens** the distribution: more probability goes to the top tokens, so output is safer, more repetitive and less diverse. As $\tau \rightarrow 0$ it approaches greedy argmax.
- $\tau = 1$ is the model's own distribution.
- $\tau > 1$ **flattens** the distribution: more probability goes to unlikely tokens, so output is more diverse but produces more errors and invented words.
- Temperature changes the probability of **every** token at **every** step.

E.3 Nucleus (top-p) sampling (Holtzman et al., 2020)

- Keep the **smallest** set V_p of highest-probability tokens whose total probability is $\geq p$, set every other token to 0 and renormalize over V_p .
- **Adaptive:** when the model is confident, the nucleus is tiny (maybe 1–2 tokens). When it is uncertain, the nucleus is large. It removes the long, low-quality **tail** without changing the order of the top tokens.
- $p = 1$ means no truncation. Smaller p is more conservative.
- Apply temperature first, then top-p on the temperature-scaled distribution.
- **Why sample a rank:** after sorting, you sample a position in the sorted list, then map it back through `sorted_indices` to the real vocabulary ID (or scatter the probabilities back to vocabulary order before sampling).

E.4 Why small models need these tricks

A 19M-parameter model assigns non-trivial total probability to many wrong tokens in the tail. Pure sampling ($\tau = 1$, $p = 1$) regularly picks one, and one bad token can derail the rest of the story. Top-p cuts off the tail, and a lower temperature concentrates probability. Both trade **diversity** against **coherence**.

Part F — Training on TinyStories (Section 7)

F.1 Fixed budget

- Architecture fixed; sequence_length = 256 fixed; exactly **N = 10,000 optimizer updates** ($s = 0 \dots 9999$); $s_c = 9999$ fixed.
- The budget is measured in **optimizer updates**, not wall-clock time, because runtime depends on hardware and implementation.
- Recommended start: $B = 16$, $n_{acc} = 16$, $\alpha_{max} = 3e-4$, $\alpha_{min} = 3e-5$, $s_w = 200$, $\beta = (0.9, 0.95)$, $\epsilon = 1e-8$, $\lambda = 0.1$, $M = 1.0$. At that setting: 65,536 tokens per update and **655,360,000** sampled tokens in total.
- Changing B or n_{acc} changes the number of tokens seen in 10,000 updates, so record B , n_{acc} , $B \cdot n_{acc}$, tokens per update and total tokens.

F.2 Controlled short-run tuning

- Short runs of 1,000–2,000 updates are informative. Keep the **same** cap, model-init seed, training-generator seed, validation seed, evaluation frequency and validation procedure for every compared run.
- **Keep $s_c = 9999$ in short runs** instead of compressing the schedule to the short cap. A short run then sees exactly the **start** of the schedule the full run will see, so it measures what the real run will do in its first 1,500 steps.
- **Ablation ladder**: change one hyperparameter from the current baseline, compare trajectories and stability, keep the better-supported choice as the new baseline, and repeat.
- Inspect train and validation loss, LR, pre-clip gradient norm, and signs of divergence or stalling.
- When comparing effective batch sizes at a fixed step cap, remember that the runs see different numbers of tokens; this is not an equal-token comparison.

F.3 Standardized final evaluation

```
fresh torch.Generator, seed 42 → 100 independent validation batches → each 16 × 256  
mean CE over the 100 batches → PPL = exp(mean CE)
```

A **fresh** generator makes the number independent of how often validation ran during training (the training-time val generator has already advanced). Everyone evaluates on the same 409,600 tokens in the same way. **Do not average per-batch perplexities.**

F.4 Final artifact (final_model.pt)

```
state = {name: t.detach().cpu().to(torch.float16) if t.is_floating_point()  
         else t.detach().cpu()  
         for name, t in model.state_dict().items()}  
torch.save(state, "final_model.pt")
```

- A plain state dict: every tensor on the CPU, floating tensors in FP16, non-float dtypes preserved. **Model weights only**: no optimizer state, RNG state, hyperparameters or next_step.
- Must contain exactly 19,272,192 values. FP16 halves the file size (38.5 MB instead of 77 MB).
- Grading may load it through adapters.get_transformer_lm, repeat the standardized validation and try other prompts.
- The zip contains only src/, tests/adapters.py, REPORT.md, report_assets/ and final_model.pt. No dataset and no full checkpoints; these must not be committed to Git either.

F.5 Why 19 million parameters? (Chinchilla)

- Hoffmann et al. (2022): for a fixed compute budget, model size and training tokens should grow together, roughly **20 tokens per parameter**.

- $19,272,192 \times 20 \approx \mathbf{385M}$ tokens. The corpus has 466.9M tokens $\approx \mathbf{24.2 \text{ corpus tokens per parameter}}$, the same broad regime.
- The final run samples 655.36M tokens $\approx \mathbf{34.0 \text{ tokens per parameter}} \approx \mathbf{1.40 \text{ corpus-equivalents}}$. These are not literal epochs: windows are sampled **with replacement**, so some regions are seen several times and others never.
- It is a design heuristic, not a guarantee: Chinchilla was fitted on much larger models and broader text, while TinyStories is narrow and synthetic.

Part G — Your approach and results (memorize these)

G.1 Implementation

- All **35 public tests passed**, including test_restrictions.py.
- Model has exactly **19,272,192** parameters; checked by loading final_model.pt into your TransformerLM with strict loading.
- Attention uses einsum 'bhrid,bhjd->bhrij' with Q grouped as $[B, h_{kv}, g, n, d_h]$; the causal mask is built with torch.tril; no repeated K/V.
- Two fixes applied for hidden-test safety: RoPE returns the input dtype; AdamW.step() runs under @torch.no_grad().
- Training used device MPS (Apple Silicon GPU); generators seeded 42 (train) and 43 (val).

G.2 Tuning study (1,500 updates each, one change from baseline)

Experiment	Val loss	Δ vs base	Final grad norm	Tokens
Baseline (3e-4, warmup 200, β_2 0.95, λ 0.1, M 1.0, eff. batch 256)	2.0805	0	0.397	98.3M
LR 1e-4	2.5505	+0.470	0.613	98.3M
LR 5e-4	1.9586	-0.122	0.316	98.3M
min LR 1e-5 / 5e-5	2.0934 / 2.0929	+0.013	0.431 / 0.452	98.3M
Warmup 100 / 400	2.0669 / 2.1058	-0.014 / +0.025	0.415 / 0.451	98.3M
β_2 0.90 / 0.999	2.0747 / 2.0667	-0.006 / -0.014	0.406 / 0.373	98.3M
Weight decay 0.01 / 0.20	2.0907 / 2.1074	+0.010 / +0.027	0.417 / 0.426	98.3M
Grad clip 0.5 / 2.0	2.0962 / 2.0943	+0.016 / +0.014	0.403 / 0.407	98.3M
Eff. batch 128 / 64	2.1675 / 2.3277	+0.087 / +0.247	0.452 / 0.510	49.2M / 24.6M

What you should be able to say about each result

- **LR**: by far the biggest effect. With a fixed update budget, a higher LR makes faster progress; 5e-4 trained stably. The main evidence is the lower validation loss; the gradient norm (0.613 vs 0.316) is supporting information only.
- **min LR**: $s_c = 9999$ was kept, so after 1,500 updates the LR had only decayed from 3e-4 to about 2.88e-4. The minimum LR barely affects the schedule this early, so these differences are noise. Kept 3e-5.
- **Warmup**: the trend was monotonic ($100 < 200 < 400$), because less warmup means more of the budget at the peak LR. The gain is small (0.014), so 100 was chosen as a low-risk adjustment.
- **β_2 0.999**: 0.014 better, within noise; a weak preference.
- **Weight decay, clipping**: no improvement; gradient norms of about 0.4 were far below $M = 1$, so clipping was rarely active. Kept 0.1 and 1.0.
- **Batch size**: confounded with tokens seen (24.6M / 49.2M / 98.3M). The comparison cannot separate the effect of batch size from the effect of less data. Kept 256.
- **Noise**: one seed per run, so differences under about 0.02 are not reliable.
- **Limitation**: one-at-a-time changes from the baseline, so the three chosen changes were never tested together (they may not add up).

G.3 Final run

Item	Value
Config	LR $5e-4 \rightarrow 3e-5$, warmup 100, s_c 9999, β (0.9, 0.999), λ 0.1, M 1.0, B $16 \times n_{acc}$ 16, seed 42, MPS
Updates / tokens	10,000 / 655,360,000 (65,536 per update)
End training loss (steps 9940–10000)	mean 1.5938, std 0.0186, no trend
End grad norm (pre-clip)	≈ 0.28 (clipping inactive)
Training-time val loss at step 10,000	1.6437
Standardized val CE	1.6114 nats/token
Perplexity	$\exp(1.6114) = 5.0100$
Train-val gap at the end	≈ 1.59 vs 1.61–1.64: no obvious gap
Limitation	the full per-step log was not saved, so there is no full loss curve

Why $1.6114 \neq 1.6437$: different validation windows. The training-time value used fewer batches from the advancing seed-43 generator; the standardized value used a fresh seed-42 generator with 100 batches. The 0.03 gap shows how much the estimate varies with the windows sampled.

G.4 Decoding study (final_model.pt, 100 new tokens, 3 prompts $\times$ 5 seeds = 15 samples per setting)

Setting	Mean tokens	EOT	Distinct-2	Repeated 4-grams
T 0.7, p 1.0	86.3	33%	0.868	2.0%
T 1.0, p 1.0	97.7	13%	0.918	0.5%
T 1.3, p 1.0	83.0	33%	0.966	0.1%
T 1.0, p 0.8	85.9	20%	0.891	1.4%
T 1.0, p 0.9	89.3	27%	0.900	1.1%

- Prompts: 'Once upon a time', 'The little girl', 'One day, a small'. These are typical TinyStories openings and are open-ended; the third stops mid-phrase to test grammatical completion.
- **Temperature:** variety rises and repetition falls as T increases, but coherence falls. T = 1.3 produced non-words ('armgo', 'complexed') and nonsense ('She loved prunes and wanted to jump perfect low'). A high distinct-2 score is **not** the same as good text.
- **Top-p:** stronger truncation lowers variety and raises repetition, but removes most surface errors. p 0.8 and 0.9 wrote the same first four sentences for prompt 1 with seed 42, which shows top-p only makes a difference where the model is uncertain.
- T = 1.0 / p = 1.0 is the shared reference point of both studies.
- Conclusion (an observation, not a universal rule): T = 1.0 with p = 0.9 gave a useful balance in these samples.

Part H — Formula sheet and glossary

H.1 Formula sheet

Quantity	Formula
Linear	$y = xW^T, W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$
Linear init σ	$\sqrt{2/(d_{\text{in}} + d_{\text{out}})}$, truncated at $\pm 3\sigma$
RMSNorm	$\gamma \odot x / \sqrt{(\text{mean}(x^2) + \epsilon)}$
SiLU	$x \cdot \sigma(x) = x/(1+e^{-x})$
SwiGLU	$W_{\text{down}}[\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x]$
RoPE frequency / angle	$\omega_k = \Theta^{-(2k-2)/d_h}, \phi_{i,k} = i \cdot \omega_k$
RoPE efficient form	$R_i x = x \odot c_i + \text{rotate_pair}(x) \odot s_i$
Stable softmax	$\exp(z - \max z) / \sum \exp(z - \max z)$
Attention	$\text{softmax}(QK^T/\sqrt{d_k} + \text{mask}) V$
Cross-entropy	$-z_y + \log \sum \exp(z_c)$
Perplexity	$\exp(\text{mean CE})$
AdamW	$m \leftarrow \beta_1 m + (1-\beta_1)g; v \leftarrow \beta_2 v + (1-\beta_2)g^2; \theta \leftarrow (1-\alpha\lambda)\theta; \theta \leftarrow \theta - \alpha \hat{m}/(\sqrt{\hat{v}}+\epsilon)$
Cosine LR	$\alpha_{\min} + \frac{1}{2}(1+\cos(\pi(s-s_w)/(s_c-s_w)))(\alpha_{\max}-\alpha_{\min})$
Clip	$g \leftarrow g \cdot M/(\ g\ +\epsilon)$ if $\ g\ > M$
Tokens / update	$B \cdot n \cdot n_{\text{acc}}$
Temperature	$\text{softmax}(z/\tau)$
Parameters	$2 V d + L[2d^2 + 2d \cdot h_{kv}d_h + 3d \cdot d_{ff} + 2d] + d$

H.2 Glossary of key terms

Term	Meaning
Decoder-only Transformer	A stack of causal self-attention + feed-forward blocks that predicts the next token (GPT-style); there is no encoder.
Logits	Unnormalized scores over the vocabulary; softmax turns them into probabilities.
Residual stream	The running d_{model} vector that every sub-layer reads from (after a norm) and adds its output back into.
Pre-norm	Normalize each sub-layer's input rather than its output; gives a clean identity path and more stable training.
RMSNorm	Scale-only normalization by the root mean square; no centering and no bias.
Gain (γ)	The learned per-feature scale in RMSNorm, initialized to 1.
SwiGLU / GLU	A gated FFN: one projection (through SiLU) gates another element-wise.
RoPE	Rotary position embedding: rotate Q/K pairs by position-dependent angles so scores depend on relative offset.
Adjacent-pair vs half-split	Two coordinate layouts for RoPE; this assignment uses adjacent pairs, and the two must not be mixed.
Non-persistent buffer	A module tensor that moves with <code>.to(device)</code> but is neither trained nor saved in the <code>state_dict</code> .
Causal mask	Lower-triangular mask: query i may see keys $j \leq i$ only.
GQA / MQA / MHA	Grouped-query attention (K/V heads shared by groups of query heads) / a single shared K/V head / one K/V head per query head.
KV cache	Stored keys and values from earlier tokens used to speed up generation; GQA makes it smaller. Not implemented in this assignment.
Untied embeddings	The input embedding and output LM head are separate parameters.
Cross-entropy (nats)	Mean negative log-likelihood using the natural log.
Perplexity	$\exp(\text{mean CE})$: the effective number of equally likely next-token choices.
Log-sum-exp trick	$\log \sum e^z = m + \log \sum e^{z-m}$, which avoids overflow.
AdamW	Adam with decoupled weight decay applied directly to the weights.
First / second moment	Running averages of the gradient and of the squared gradient.
Bias correction	Division by $(1-\beta^t)$ that fixes the zero-initialized moment estimates.
Decoupled weight decay	$\theta \leftarrow (1-\alpha\lambda)\theta$, applied separately from the adaptive gradient step.
Warmup	A linear LR ramp from 0 to α_{max} over s_w steps.
Cosine annealing	A smooth LR decay from α_{max} to α_{min} following half a cosine.
Global gradient clipping	Rescale all gradients by one factor so the global L2 norm is at most M .
np.memmap	Memory-mapped file access: data is loaded on demand, not all at once.
torch.Generator	An explicit RNG object with its own state, used for reproducible sampling.
Gradient accumulation	Sum the gradients of several microbatches (each loss divided by n_{acc}) before one update.
Global step / optimizer update	One call of <code>optimizer.step()</code> ; the unit of the budget and the schedule.
Effective batch size	$B \times n_{\text{acc}}$ sequences per update.
next_step	The global step to run after loading a checkpoint ($s+1$ after completing step s).
model.eval() vs inference_mode()	Evaluation behaviour of modules vs disabling autograd tracking.
Temperature	Divide logits by τ : $\tau < 1$ sharpens, $\tau > 1$ flattens.
Nucleus (top-p) sampling	Sample only from the smallest set of tokens whose cumulative probability is $\geq p$.

Term	Meaning
Distinct-2	Unique bigrams divided by total bigrams: a variety metric.
Ablation	Change one component or setting at a time to measure its effect.
Confound	A second variable that changes together with the one being tested (batch size and tokens seen).
Chinchilla scaling	Compute-optimal training uses roughly 20 tokens per parameter.
FP16	16-bit float: half the memory of FP32, with lower precision and range.
Byte-level BPE	A subword tokenizer over UTF-8 bytes, so there are no unknown tokens.
EOT	< endoftext >: separates stories in the stream and stops generation.

Part I — Quiz question bank

87 questions, grouped by part. Try each one before checking Part J. Questions marked with options are multiple choice; the rest are short-answer or calculation questions.

A. General and setup

Q1. What fraction of the PA1 grade comes from unit tests, and what does 'hidden tests' imply for your implementation?

Q2. Which of these may you use in your implementation?

- (a) `torch.nn.functional.softmax`
- (b) `torch.nn.Linear`
- (c) `torch.nn.init.trunc_normal_`
- (d) `torch.optim.AdamW`

Q3. What is the role of `tests/adapters.py`, and why may it not contain any maths?

Q4. Why is byte-level BPE used, and why don't you need an 'unknown token'?

Q5. What does the EOT token do (a) in the training stream and (b) during generation?

B. Architecture

Q6. The tests give you a weight of shape $(d_{\text{out}}, d_{\text{in}})$. Write the forward computation of your Linear layer and explain the orientation.

Q7. Compute the standard deviation used to initialize W_{gate} ($d_{\text{model}} = 512$, $d_{\text{ff}} = 1344$).

Q8. Give two differences between RMSNorm and LayerNorm, and one reason to prefer RMSNorm.

Q9. Why does RMSNorm upcast float16/bfloat16 inputs to float32, and what must it return?

Q10. Why does a pre-norm Transformer need a final RMSNorm after the last block, while post-norm does not?

Q11. Show why $d_{\text{ff}} \approx (8/3) \cdot d_{\text{model}}$ keeps SwiGLU's parameter count equal to a standard 4d FFN, and why 1344 rather than 1365?

Q12. In RoPE, which tensors are rotated?

- (a) Q, K and V
- (b) Q and K only
- (c) K and V only
- (d) The residual stream before attention

Q13. Prove that RoPE makes the attention score depend only on relative position.

Q14. For $\text{head_dim} = 32$ and $\Theta = 10,000$, what are the frequencies of the first and last pairs? What shape are the cos/sin buffers for `context_length 256`?

Q15. Why are the RoPE cos/sin tables registered with `persistent=False` rather than as `nn.Parameters`?

Q16. What is the difference between the adjacent-pair and half-split RoPE conventions, and why does it matter for grading?

Q17. Why subtract the maximum before exponentiating in softmax? Why is the result unchanged?

Q18. Why are attention scores divided by $\sqrt{d_k}$?

- Q19.** In this assignment's mask convention, what does True mean, and what happens if a query row is all False?
- Q20.** Give the shapes of W_Q , W_K , W_V and W_O for the final model.
- Q21.** What is the group size g in the final model, and which KV head does query head 7 (1-indexed) use?
- Q22.** Why must you NOT use `repeat_interleave` to copy K and V for each query head in GQA?
- Q23.** What are two benefits of GQA over MHA? What does MQA sacrifice?
- Q24.** After attention, why does the order in which you merge the (h, r) axes matter?
- Q25.** What must the LM do if it receives a sequence of length 300 with `context_length = 256`? And length 0?
- Q26.** Derive the parameter count of the final model.
- Q27.** What fraction of the parameters is in the embedding and LM head? What does that suggest?
- Q28.** What would happen without the causal mask during training?

C. Loss and optimization

- Q29.** Write the cross-entropy for one position and explain why you should not compute `log(softmax(logits))`.
- Q30.** What loss should a freshly initialized model have on this vocabulary, and what perplexity is that?
- Q31.** Your final mean validation CE is 1.6114. Compute the perplexity and interpret it correctly.
- Q32.** Why must you not average per-batch perplexities?
- Q33.** Two batches have CE 1.5 and 1.7. Compare the correct PPL with the average of the per-batch PPLs.
- Q34.** Explain Adam's bias correction. What would go wrong without it at step 1?
- Q35.** What is 'decoupled' about AdamW's weight decay compared with L2 regularization in Adam?
- Q36.** Why does AdamW keep a per-parameter step count t instead of using the global step?
- Q37.** What must AdamW do with a parameter whose grad is None?
- Q38.** Why is the whole `AdamW.step()` wrapped in `torch.no_grad()`, and how is a closure still evaluated with gradients?
- Q39.** Why does AdamW re-validate lr , β s, ϵ s and `weight_decay` inside `step()`?
- Q40.** With $\alpha_{\max} = 5e-4$, $\alpha_{\min} = 3e-5$, $s_w = 100$, $s_c = 9999$: what is the LR at $s = 0$, $s = 50$, $s = 100$, $s = 5049.5$ and $s = 9999$?
- Q41.** Why use warmup at all?
- Q42.** Why is global-norm clipping preferred to clipping each parameter tensor separately?
- Q43.** The global gradient norm is 2.5 and $M = 1.0$. What happens to the gradients, and what does your function return?

Q44. In your final run, the gradient norm near the end was about 0.28 with $M = 1.0$. What does that tell you?

D. Training infrastructure

Q45. Why store token IDs as uint16, and why read them with np.memmap?

Q46. Why must load_token_array reject a file with an odd byte length?

Q47. Why is the exclusive upper bound for start positions $N - n$ rather than $N - n + 1$?

Q48. Why read $n + 1$ tokens per window?

Q49. Why pass an explicit torch.Generator to get_batch instead of calling torch.manual_seed?

Q50. Why does the training script use separate training and validation generators?

Q51. List everything a checkpoint must contain. Why is the optimizer state essential?

Q52. You complete global step $s = 4,999$ and save a checkpoint. What value of next_step is stored?

Q53. $B = 16$, $n = 256$, $n_acc = 16$. Tokens per update? Effective batch size? Total tokens in 10,000 updates?

Q54. Why is each microbatch loss divided by n_acc before backward()?

Q55. In gradient accumulation, how many times per global step do you call zero_grad, clip and optimizer.step?

Q56. Validation, logging and checkpointing are all due at the same step. In what order, and why is checkpointing last?

Q57. What is the difference between model.eval() and torch.inference_mode()? Which does validation use?

Q58. What sanity check does the manual recommend before the long run, and what result proves correctness?

Q59. Why are sequences allowed to cross story boundaries?

E. Generation

Q60. Why must the prompt be non-empty?

Q61. The sequence has grown to 300 tokens, context_length = 256. What does generate() feed to the model?

Q62. What does temperature $\tau \rightarrow 0$ approach? What does $\tau = 1.3$ do?

Q63. Probabilities sorted descending are [0.5, 0.3, 0.15, 0.05]. Which tokens survive top-p with $p = 0.8$? With $p = 0.85$? What are the renormalized probabilities for $p = 0.8$?

Q64. Why is top-p called 'adaptive' compared with top-k?

Q65. In your generate(), you sample a rank from the sorted distribution. How do you get the actual token ID?

Q66. Must EOT be included in the returned sequence?

Q67. Why does generation recompute the whole context each step, and what would fix that?

Q68. Why is generate() required to check context_length == model.context_length?

F. Training on TinyStories

Q69. Why is the training budget defined in optimizer updates rather than wall-clock time?

Q70. Why keep $s_c = 9999$ in 1,500-step tuning runs instead of setting $s_c = 1,500$?

Q71. What is an ablation ladder?

Q72. Why is comparing effective batch sizes at a fixed update cap not an equal-token comparison?

Q73. Why must the final evaluation use a fresh generator seeded with 42?

Q74. List four requirements for `final_model.pt`.

Q75. Using Chinchilla's heuristic, how many tokens would be compute-optimal for this model, and how many did the final run sample per parameter?

Q76. Why is 655M sampled tokens NOT '1.4 epochs'?

G. Your approach and results

Q77. Which hyperparameter had the largest effect in your tuning, and what evidence supports your choice?

Q78. Your min-LR runs ($1e-5$, $5e-5$) barely differed from the baseline. Why can't that experiment show an effect at 1,500 updates?

Q79. Why did you choose `warmup = 100` even though the gain was only 0.014?

Q80. Why didn't you reduce the effective batch size even though it would allow more updates of a different kind?

Q81. What is the main limitation of your tuning method?

Q82. Your training-time final val loss was 1.6437 but the standardized value is 1.6114. Why the difference, and which do you report?

Q83. What does a final train loss of about 1.59 and val loss of 1.61-1.64 tell you, and what can't it tell you?

Q84. In your decoding study, $T = 1.3$ had the highest distinct-2 (0.966). Does that mean it produced the best text? Explain.

Q85. Why did top-p 0.8 and 0.9 generate the same first four sentences for the same prompt and seed?

Q86. Which setting did you conclude gave a useful balance, and how did you qualify the claim?

Q87. Name the two implementation fixes you made after the public tests passed, and why they mattered.

Part J — Answer key

A. General and setup

Q1. What fraction of the PA1 grade comes from unit tests, and what does 'hidden tests' imply for your implementation?

70% from unit tests (30% from REPORT.md). Hidden tests use the same adapters but can check any behaviour documented in the manual, so an implementation must follow every stated contract (dtypes, validations, rejections), not just pass the public tests.

Q2. Which of these may you use in your implementation?

(c). `torch.nn.init.trunc_normal_` is explicitly allowed; the others are built-in versions of deliverables.

Q3. What is the role of tests/adapters.py, and why may it not contain any maths?

It is glue between the test harness and your code: it imports your class, constructs it, loads supplied weights and calls it. Keeping maths out of it ensures the graded logic lives in `src/`, so the tests actually test your implementation.

Q4. Why is byte-level BPE used, and why don't you need an 'unknown token'?

It operates on UTF-8 bytes, so every possible string is representable from 256 byte tokens; merges then shorten frequent sequences. Nothing can be out-of-vocabulary.

Q5. What does the EOT token do (a) in the training stream and (b) during generation?

(a) It separates concatenated stories, so the model learns where a story ends and a new one begins. (b) Generation stops immediately after the model produces it (it is kept in the output).

B. Architecture

Q6. The tests give you a weight of shape (d_out, d_in). Write the forward computation of your Linear layer and explain the orientation.

$y = x @ W$. PyTorch stores W as (out_features, in_features); with feature-last inputs of shape $(..., d_{in})$, multiplying by W^T gives $(..., d_{out})$. Storing W^T as the parameter would break the tests' weight injection.

Q7. Compute the standard deviation used to initialize W_{gate} ($d_{model} = 512$, $d_{ff} = 1344$).

$\sigma = \sqrt{(2/(512+1344))} = \sqrt{(2/1856)} = \sqrt{0.0010776} \approx 0.0328$, truncated at $\pm 3\sigma \approx \pm 0.0985$.

Q8. Give two differences between RMSNorm and LayerNorm, and one reason to prefer RMSNorm.

RMSNorm does not subtract the mean (no centering) and has no bias, only a gain γ . It is cheaper to compute and works as well in practice (LLaMA uses it).

Q9. Why does RMSNorm upcast float16/bfloat16 inputs to float32, and what must it return?

Squaring and averaging small values in low precision can underflow or lose accuracy; float32 keeps the RMS accurate. It must cast the result back and return the input's original dtype (and shape).

Q10. Why does a pre-norm Transformer need a final RMSNorm after the last block, while post-norm does not?

In pre-norm the residual stream itself is never normalized (only sub-layer inputs are), so its scale can grow. The final norm normalizes it before the LM head. In post-norm, every block output is already normalized.

Q11. Show why $d_{\text{ff}} \approx (8/3) \cdot d_{\text{model}}$ keeps SwiGLU's parameter count equal to a standard 4d FFN, and why 1344 rather than 1365?

Standard: 2 matrices $\times d \times 4d = 8d^2$. SwiGLU: 3 matrices $\times d \times d_{\text{ff}} = 8d^2 \Rightarrow d_{\text{ff}} = 8d/3 \approx 1365.3$. It is rounded to a multiple of 64 for hardware efficiency: $1344 = 21 \times 64$.

Q12. In RoPE, which tensors are rotated?

(b). RoPE rotates queries and keys only; values are never rotated.

Q13. Prove that RoPE makes the attention score depend only on relative position.

Rotations satisfy $R(\phi_i)^T R(\phi_j) = R(\phi_j - \phi_i)$. So $(R_i q)^T (R_j k) = q^T R_i^T R_j k = q^T R((j-i)\omega) k$. The score depends on the offset $j - i$, not on i and j separately.

Q14. For $\text{head_dim} = 32$ and $\Theta = 10,000$, what are the frequencies of the first and last pairs? What shape are the cos/sin buffers for context_length 256?

Pair 1: $\omega_1 = \Theta^0 = 1$ rad/position (fastest). Pair 16: $\omega_{16} = 10000^{(-30/32)} \approx 1.78e-4$ (slowest).
Buffers: (256, 16), i.e. (context_length, head_dim/2).

Q15. Why are the RoPE cos/sin tables registered with persistent=False rather than as nn.Parameters?

They are fixed, not learned, so they must not be Parameters (the optimizer would update them). As buffers they move with `.to(device)`. Non-persistent keeps them out of the state_dict, since they can be recomputed, and keeps final_model.pt at exactly 19,272,192 values.

Q16. What is the difference between the adjacent-pair and half-split RoPE conventions, and why does it matter for grading?

Adjacent pairs rotate $(x_1, x_2), (x_3, x_4), \dots$; half-split (GPT-NeoX) pairs x_i with $x_{\{i+d/2\}}$. They are the same family of rotations in a different layout, and they give different numbers for the same weights. The tests use adjacent pairs, so using half-split fails them.

Q17. Why subtract the maximum before exponentiating in softmax? Why is the result unchanged?

exp of large values overflows to inf and $\text{inf}/\text{inf} = \text{NaN}$. $\text{softmax}(z) = \text{softmax}(z - c)$ for any constant c , because the factor $e^{(-c)}$ cancels between numerator and denominator. Choosing $c = \max z$ makes every exponent ≤ 0 .

Q18. Why are attention scores divided by $\sqrt{d_k}$?

With unit-variance independent entries, $q^T k$ has variance d_k . Large scores saturate softmax (nearly one-hot, tiny gradients). Dividing by $\sqrt{d_k}$ keeps the score variance around 1.

Q19. In this assignment's mask convention, what does True mean, and what happens if a query row is all False?

True = the query may attend to that key. An all-False row would make $\text{softmax } 0/0 = \text{NaN}$, so the implementation must detect it and raise an error rather than return non-finite values.

Q20. Give the shapes of W_Q , W_K , W_V and W_O for the final model.

W_Q : $(16 \cdot 32, 512) = (512, 512)$. W_K, W_V : $(4 \cdot 32, 512) = (128, 512)$. W_O : $(512, 16 \cdot 32) = (512, 512)$.

Q21. What is the group size g in the final model, and which KV head does query head 7 (1-indexed) use?

$g = 16/4 = 4$. Using $a = (h-1)g + r$: $a = 7 \Rightarrow h = 2, r = 3$, so it uses KV head 2.

Q22. Why must you NOT use repeat_interleave to copy K and V for each query head in GQA?

The manual forbids materializing repeated K/V. Repeating wastes memory and compute. Arranging Q as $[B, h_{\text{kv}}, g, n, d_h]$ lets einsum broadcast each K/V head across its group.

Q23. What are two benefits of GQA over MHA? What does MQA sacrifice?

Fewer K/V projection parameters (393,216 fewer per block here) and a smaller KV cache and less memory traffic at inference (4× here), with quality close to MHA. MQA shares one K/V head, the smallest cache, but tends to lose more quality.

Q24. After attention, why does the order in which you merge the (h, r) axes matter?

W_O expects the query heads in the original projection order $a = (h-1)g + r$. Merging in another order feeds W_O permuted heads, so the output is wrong even though the shapes are correct.

Q25. What must the LM do if it receives a sequence of length 300 with context_length = 256? And length 0?

Raise an error in both cases: the length must satisfy $1 \leq n \leq \text{context_length}$. Cropping is generate()'s job, not the model's.

Q26. Derive the parameter count of the final model.

$2 \cdot 8192 \cdot 512 = 8,388,608$ (embedding + head). Per block: $2 \cdot 512^2 = 524,288$ (Q, O) + $2 \cdot 512 \cdot 128 = 131,072$ (K, V) + $3 \cdot 512 \cdot 1344 = 2,064,384$ (SwiGLU) + 1,024 (two norms) = 2,720,768; $\times 4 = 10,883,072$. + 512 (final norm). Total = 19,272,192.

Q27. What fraction of the parameters is in the embedding and LM head? What does that suggest?

$8,388,608 / 19,272,192 \approx 43.5\%$. In a small model with an 8K vocabulary, the vocabulary matrices are a large share; most of the 'thinking' capacity sits in only ~10.9M block parameters.

Q28. What would happen without the causal mask during training?

Each position could attend to future tokens, including the target itself. The model would learn to copy the answer, training loss would become artificially low, and generation (where the future is unknown) would fail.

C. Loss and optimization

Q29. Write the cross-entropy for one position and explain why you should not compute $\log(\text{softmax}(\text{logits}))$.

$\ell = -z_y + \log \text{sumexp}(z)$. A full log-softmax tensor is $B \times n \times |V|$ extra memory, and log of a probability that underflowed to 0 gives $-\text{inf}$. Log-sum-exp with a max shift plus gathering only z_y is stable and cheap.

Q30. What loss should a freshly initialized model have on this vocabulary, and what perplexity is that?

About $\ln 8192 \approx 9.01$ nats/token (a near-uniform prediction), so perplexity $\approx 8,192$.

Q31. Your final mean validation CE is 1.6114. Compute the perplexity and interpret it correctly.

$\exp(1.6114) \approx 5.01$. On average, the model's next-token uncertainty equals choosing among about 5 equally likely tokens. This is an effective count, not a literal uniform distribution over 5 tokens.

Q32. Why must you not average per-batch perplexities?

Perplexity is defined as $\exp$ of the mean CE. $\exp$ is convex, so $\text{mean}(\exp(\ell_b)) \geq \exp(\text{mean } \ell_b)$ (Jensen): averaging perplexities overestimates it and over-weights hard batches.

Q33. Two batches have CE 1.5 and 1.7. Compare the correct PPL with the average of the per-batch PPLs.

Correct: $\exp(1.6) = 4.953$. Wrong: $(e^{1.5} + e^{1.7})/2 = (4.482 + 5.474)/2 = 4.978$, which is higher, as Jensen predicts.

Q34. Explain Adam's bias correction. What would go wrong without it at step 1?

m and v start at 0, so after step 1, $m = (1-\beta_1)g$ and $v = (1-\beta_2)g^2$, both biased toward 0. Dividing by $(1-\beta^t)$ recovers $\hat{m} = g$ and $\hat{v} = g^2$. Without it ($\beta_1 = 0.9$, $\beta_2 = 0.999$): $m = 0.1g$ and $\sqrt{v} \approx 0.0316|g|$, so the first step would be about $0.1/0.0316 \approx 3.16 \times \alpha$ per coordinate instead of α . Early steps would be mis-scaled (too large here), exactly when the model is most fragile.

Q35. What is 'decoupled' about AdamW's weight decay compared with L2 regularization in Adam?

L2 adds $\lambda\theta$ to the gradient, which then gets divided by $\sqrt{\hat{v}}$, so parameters with large gradient history are decayed less. AdamW applies $\theta \leftarrow (1-\alpha\lambda)\theta$ directly, independent of the adaptive scaling, so all weights decay at the same rate.

Q36. Why does AdamW keep a per-parameter step count t instead of using the global step?

Some parameters may not receive a gradient on some steps (grad None). Their moments were updated fewer times, so their bias correction must use their own update count.

Q37. What must AdamW do with a parameter whose grad is None?

Skip it completely: no weight decay, no moment update, no step-count increment.

Q38. Why is the whole `AdamW.step()` wrapped in `torch.no_grad()`, and how is a closure still evaluated with gradients?

Parameter updates must not be recorded in an autograd graph (it would waste memory and connect iterations). The closure is called inside `torch.enable_grad()`, which re-enables gradients just for the loss computation.

Q39. Why does AdamW re-validate `lr`, `betas`, `eps` and `weight_decay` inside `step()`?

Callers (for example the LR scheduler) can change a param group's values after construction, so `step()` must read and validate the current values each time.

Q40. With $\alpha_{\text{max}} = 5e-4$, $\alpha_{\text{min}} = 3e-5$, $s_w = 100$, $s_c = 9999$: what is the LR at $s = 0$, $s = 50$, $s = 100$, $s = 5049.5$ and $s = 9999$?

$s=0$: 0. $s=50$: $2.5e-4$. $s=100$: $5e-4$. $s=5049.5$ (cosine midpoint): $(5e-4 + 3e-5)/2 = 2.65e-4$. $s=9999$: $3e-5$.

Q41. Why use warmup at all?

At the start, the weights are random and Adam's second-moment estimates are based on very few gradients, so large steps can diverge. Warmup ramps the LR gradually until the statistics stabilize.

Q42. Why is global-norm clipping preferred to clipping each parameter tensor separately?

Scaling every gradient by the same factor keeps the direction of the full update and only shortens it. Per-tensor clipping changes the relative sizes, and therefore the direction.

Q43. The global gradient norm is 2.5 and $M = 1.0$. What happens to the gradients, and what does your function return?

Every gradient is multiplied by $1.0/(2.5 + 1e-6) \approx 0.4$. The function returns 2.5, the pre-clipping norm, as a Python float.

Q44. In your final run, the gradient norm near the end was about 0.28 with $M = 1.0$. What does that tell you?

Clipping was not active at the end of training: updates were not being limited, and training was stable.

D. Training infrastructure

Q45. Why store token IDs as uint16, and why read them with np.memmap?

8,192 < 65,536, so 2 bytes per token is enough. memmap loads only the pages you access, so the 467M-token file (≈ 934 MB) never has to be fully in RAM; converting it to int64 would need about 3.7 GB.

Q46. Why must load_token_array reject a file with an odd byte length?

Each token is exactly 2 bytes; an odd length means a truncated or corrupt file (or the wrong format).

Q47. Why is the exclusive upper bound for start positions $N - n$ rather than $N - n + 1$?

The largest start is then $N - n - 1$, and its $n + 1$ window ends at index $N - 1$ (the last token). A start of $N - n$ would need index N , past the end of the stream.

Q48. Why read $n + 1$ tokens per window?

The inputs are the first n tokens and the targets are the same window shifted by one; n tokens alone would leave the last input without a target.

Q49. Why pass an explicit torch.Generator to get_batch instead of calling torch.manual_seed?

An explicit generator gives reproducible, independent random streams without affecting global RNG state; the same seed gives the same batch sequence, and its state can be saved in checkpoints.

Q50. Why does the training script use separate training and validation generators?

So that validation sampling never consumes random numbers from the training stream. Changing eval_interval or the number of validation batches does not change which training batches are drawn.

Q51. List everything a checkpoint must contain. Why is the optimizer state essential?

Model state_dict, optimizer state_dict, next_step, and both generator states. The optimizer state holds Adam's moments and step counts; without them, resuming resets momentum and bias correction and changes the trajectory.

Q52. You complete global step $s = 4,999$ and save a checkpoint. What value of next_step is stored?

$\text{next_step} = s + 1 = 5,000$.

Q53. $B = 16$, $n = 256$, $n_acc = 16$. Tokens per update? Effective batch size? Total tokens in 10,000 updates?

$16 \cdot 256 \cdot 16 = 65,536$ tokens/update. Effective batch = $16 \cdot 16 = 256$ sequences. Total = 655,360,000 tokens.

Q54. Why is each microbatch loss divided by n_acc before backward()?

Gradients add up across backward() calls. Dividing each loss by n_acc makes the accumulated gradient equal to the gradient of the mean loss over the effective batch, so it matches a single large batch and is independent of n_acc .

Q55. In gradient accumulation, how many times per global step do you call zero_grad, clip and optimizer.step?

Once each: zero_grad at the start, then after all n_acc backward passes clip once and step once.

Q56. Validation, logging and checkpointing are all due at the same step. In what order, and why is checkpointing last?

Validate $\rightarrow$ log $\rightarrow$ checkpoint. Checkpointing last ensures the saved validation-generator state already includes that validation's batches, so a resumed run continues exactly as an uninterrupted one.

Q57. What is the difference between `model.eval()` and `torch.inference_mode()`? Which does validation use?

`eval()` switches modules to evaluation behaviour (e.g. dropout off) but still tracks gradients; `inference_mode()` disables autograd bookkeeping. Validation uses both, then restores the previous mode with `model.train(was_training)`.

Q58. What sanity check does the manual recommend before the long run, and what result proves correctness?

Repeatedly train a small model on one fixed minibatch; a correct model, loss, backward and optimizer drive that loss down sharply (it overfits). If it doesn't, something in the pipeline is broken.

Q59. Why are sequences allowed to cross story boundaries?

Sampling windows from one continuous stream avoids padding and wasted compute; the EOT token marks the boundary, so the model learns to end a story and begin a new one.

E. Generation

Q60. Why must the prompt be non-empty?

The next-token distribution is read from the logits at the last input position; with no input there is no position to read from.

Q61. The sequence has grown to 300 tokens, `context_length = 256`. What does `generate()` feed to the model?

Only the most recent 256 tokens (a sliding window). The full sequence is kept and returned.

Q62. What does temperature $\tau \rightarrow 0$ approach? What does $\tau = 1.3$ do?

$\tau \rightarrow 0$ approaches greedy argmax (deterministic). $\tau = 1.3$ flattens the distribution, giving more weight to unlikely tokens: more diverse, less coherent.

Q63. Probabilities sorted descending are [0.5, 0.3, 0.15, 0.05]. Which tokens survive top-p with $p = 0.8$? With $p = 0.85$? What are the renormalized probabilities for $p = 0.8$?

$p = 0.8$: cumulative 0.5, 0.8 $\rightarrow$ the first two ($0.8 \geq 0.8$), renormalized to [0.625, 0.375]. $p = 0.85$: 0.5, 0.8, 0.95 $\rightarrow$ the first three (0.95 is the first cumulative ≥ 0.85).

Q64. Why is top-p called 'adaptive' compared with top-k?

The number of kept tokens changes every step: a confident step keeps very few tokens, an uncertain step keeps many. Top-k always keeps k tokens, regardless of the model's confidence.

Q65. In your `generate()`, you sample a rank from the sorted distribution. How do you get the actual token ID?

`next_token = sorted_indices[sampled_rank]`: the sort permutation maps the rank back to the original vocabulary index.

Q66. Must EOT be included in the returned sequence?

Yes. Stop immediately after generating it and keep it in the output.

Q67. Why does generation recompute the whole context each step, and what would fix that?

No KV cache is implemented, so each new token needs a full forward pass over the cropped context. A KV cache that stores past keys and values would make each step process only the new token (and GQA would make that cache 4× smaller).

Q68. Why is `generate()` required to check `context_length == model.context_length`?

So generation cannot silently crop to a window different from the one the model supports (and was trained with), which could cause errors or wrong behaviour.

F. Training on TinyStories

Q69. Why is the training budget defined in optimizer updates rather than wall-clock time?

Runtime depends on hardware and implementation, so a fixed update count makes runs comparable across students and machines.

Q70. Why keep $s_c = 9999$ in 1,500-step tuning runs instead of setting $s_c = 1,500$?

So the short run follows exactly the first 1,500 steps of the schedule the real run will use. A compressed schedule would decay to $\alpha_{\min}$ quickly and measure a different training regime.

Q71. What is an ablation ladder?

Change one hyperparameter from the current baseline, compare, adopt the better-supported value as the new baseline, and repeat, so each comparison isolates one change.

Q72. Why is comparing effective batch sizes at a fixed update cap not an equal-token comparison?

Tokens seen = updates $\times$ tokens per update. A smaller effective batch processes fewer tokens in the same number of updates, so a worse loss may just reflect less data.

Q73. Why must the final evaluation use a fresh generator seeded with 42?

It makes the result independent of how often validation ran during training (which advances the training-time generator), and everyone is evaluated on the same 100 batches in the same way.

Q74. List four requirements for `final_model.pt`.

A plain state dict of the final model; all tensors on the CPU; floating tensors in FP16 (non-float dtypes preserved); model weights only (no optimizer or RNG state, hyperparameters or `next_step`); exactly 19,272,192 values; the same model used for the evaluation and decoding study.

Q75. Using Chinchilla's heuristic, how many tokens would be compute-optimal for this model, and how many did the final run sample per parameter?

$\approx 20 \times 19.27\text{M} \approx 385\text{M}$ tokens. The run sampled $655.36\text{M} \approx 34$ tokens per parameter.

Q76. Why is 655M sampled tokens NOT '1.4 epochs'?

Windows are drawn at random with replacement, so some regions are seen several times and others never. 1.4 is sampled token exposure relative to the corpus size, not a count of complete passes.

G. Your approach and results

Q77. Which hyperparameter had the largest effect in your tuning, and what evidence supports your choice?

The peak LR: $1\text{e-}4$ gave 2.5505, the baseline $3\text{e-}4$ gave 2.0805 and $5\text{e-}4$ gave 1.9586 (-0.122). The main evidence is the much lower validation loss; the run was stable. Gradient norms (0.613 vs 0.316) are supporting information only.

Q78. Your min-LR runs ($1\text{e-}5$, $5\text{e-}5$) barely differed from the baseline. Why can't that experiment show an effect at 1,500 updates?

With $s_c = 9999$, after 1,500 updates the LR had only decayed from $3\text{e-}4$ to about $2.88\text{e-}4$; $\alpha_{\min}$ barely influences the schedule this early, so differences of about 0.013 are noise.

Q79. Why did you choose `warmup = 100` even though the gain was only 0.014?

The trend was monotonic across $100 < 200 < 400$ (less warmup = more budget at peak LR), training remained stable, and it is a low-risk change. It is still a weak, not a strong, result.

Q80. Why didn't you reduce the effective batch size even though it would allow more updates of a different kind?

At a fixed 10,000 updates a smaller batch sees proportionally fewer tokens (24.6M/49.2M vs 98.3M in tuning), and the results were worse. The comparison is confounded, so nothing justified reducing it.

Q81. What is the main limitation of your tuning method?

Each change was tested alone against the same baseline with one seed, so the combination (LR $5e-4$ + warmup 100 + β_2 0.999) was never tested jointly, and differences under about 0.02 cannot be separated from noise.

Q82. Your training-time final val loss was 1.6437 but the standardized value is 1.6114. Why the difference, and which do you report?

Different validation windows: the training-time value used fewer batches from the advancing seed-43 generator, the standardized value a fresh seed-42 generator over 100 batches. Report the standardized one; the gap shows sampling variance.

Q83. What does a final train loss of about 1.59 and val loss of 1.61-1.64 tell you, and what can't it tell you?

There is no obvious train-validation gap at the end of training. Without the full curves it cannot show how the gap evolved during training.

Q84. In your decoding study, $T = 1.3$ had the highest distinct-2 (0.966). Does that mean it produced the best text? Explain.

No. Much of the extra variety came from wrong or invented tokens ('armgo', 'complexed') and incoherent sentences. Variety metrics reward randomness as much as creativity; coherence must be judged separately.

Q85. Why did top-p 0.8 and 0.9 generate the same first four sentences for the same prompt and seed?

When the model is confident, both nuclei contain the same few tokens and the same random draw selects the same token; they diverge only at uncertain steps. This shows top-p's adaptive nature.

Q86. Which setting did you conclude gave a useful balance, and how did you qualify the claim?

$T = 1.0$, top-p = 0.9: variety 0.900 (close to 0.918 untruncated), repetition 1.1% (below $T = 0.7$'s 2.0%) and grammatical samples. It is an observation from 15 samples per setting with manual coherence judgement, not a universal rule.

Q87. Name the two implementation fixes you made after the public tests passed, and why they mattered.

RoPE returning the input dtype (it returned float32 for fp16/bf16 inputs); @torch.no_grad() on AdamW.step() (the moment updates could build an autograd graph). Both are explicit manual requirements that hidden tests could check.